



Calendrix

ENTREGA FINAL

v1.0.0

1. Introducción	2
2. Modelo Computacional	2
2.1. Dominio	2
2.2. Lenguaje	3
3. Implementación	4
3.1. Frontend	4
3.2. Backend	4
3.3. Dificultades encontradas	5
4. Futuras extensiones	5
5. Conclusiones	6
6. Referencias	6

INTEGRANTES

Nombre	Apellido	Legajo	E-mail
Julieta	Techenski	62547	jtechenski@itba.edu.ar
Matias	Coleur	63461	mcoleur@itba.edu.ar
Valentina	Marti Reta	63225	vmartireta@itba.edu.ar

1. Introducción

En un mundo donde la gestión eficiente del tiempo es esencial tanto en el ámbito personal como en el empresarial, organizar horarios puede convertirse en un desafío , especialmente cuando involucra a múltiples personas y equipos. Desde compromisos laborales hasta actividades personales, la falta de herramientas de planificación y organización puede dar lugar al caos rápidamente.

Con el objetivo de abordar esta necesidad, se ha desarrollado un lenguaje declarativo diseñado específicamente para la organización de horarios. Este lenguaje permite la creación y gestión de usuarios, tareas, eventos y equipos. A partir de las especificaciones declaradas, es posible generar calendarios semanales, mensuales o anuales en formato JavaScript y HTML, proporcionando soluciones visuales y prácticas para la planificación y coordinación.

Esta herramienta busca ofrecer una solución adaptable y flexible, permitiendo a los usuarios modelar eficiente y fácilmente sus necesidades de organización. Gracias a su capacidad de integrarse con facilidad a sistemas y formatos modernos, gracias al código fuente que genera, este lenguaje se convierte en una pieza clave para optimizar el tiempo de manera efectiva.

2. Modelo Computacional

2.1. Dominio

El dominio del proyecto consiste en desarrollar un lenguaje declarativo específicamente para organizar horarios. Este lenguaje está diseñado para permitir la organización de tareas, asignación de usuarios, y la planificación de actividades en función de fechas, horarios y prioridades, proporcionando una solución flexible y adaptable para una amplia gama de necesidades de planificación y organización. Este debe de contar con una sintaxis intuitiva y fácil de usar, tanto para entender cómo para programar. Cada tarea o evento se define especificando su tipo y los parámetros correspondientes, como lo son título, fecha de inicio y fin, y asignación de usuarios, entre otras cosas. A su vez, los usuarios podrán agrupar declaraciones en bloques y asignar tareas/eventos a grupos específicos.

La salida del compilador será en **JavaScript** y **HTML**, creando un archivo por cada calendario que se desee generar, sea del tipo semanal, mensual o anual. Estos lenguajes son fácilmente accesibles y pueden integrarse de manera directa en aplicación web o sistema de

gestión, brindando una representación clara y dinámica de la organización de las tareas o eventos. Para ello, se utiliza la librería **FullCalendar**, que nos permite generar las variantes de calendarios con los eventos seleccionados.

A su vez, este lenguaje permite importar información de otros archivos escritos en este lenguaje, asegurando una integración fluida con otros archivos creados en el mismo sistema. Esto facilitará la gestión y el intercambio de información entre diferentes miembros del equipo, garantizando la compatibilidad con los archivos del mismo lenguaje.

La correcta implementación de este lenguaje facilita la creación rápida, eficiente y clara de horarios, eliminando la necesidad de utilizar herramientas de planificación complicadas. Esto simplifica el procesos de planificación y organización, permitiendo personalizar calendarios de acuerdo con distintos tipos de necesidades.

2.2. Lenguaje

El lenguaje ofrece las siguientes funcionalidades:

- Se podrán definir usuarios con atributos personalizados, como nombre, rol, disponibilidad horaria y grupo de trabajo, mediante la declaración `create user`.
- Se podrán definir grupos a los cuales se le pueden asignar miembros mediante la sentencia `create group`.
- Se facilitará la agrupación de usuarios en grupos, lo cual es especialmente útil para contextos empresariales. Esto se realizará mediante la declaración `add user`.
- Se proporcionan comandos para la creación de tareas, especificando detalles como el nombre de la tarea, descripción, duración, fecha de inicio y fin con la sentencia `task`.
- Soportará reglas de negocio personalizables, como la asignación de tareas a usuarios o grupos específicos. Esto se realiza en la declaración de la misma.
- Permitirá al programador poder importar información en formato TXT para poder ser utilizada en otros programas mediante la declaración `import`.
- Se podrán crear bloques de código en el archivo principal, pero no podrán estar anidados. Esto se realizará mediante la sentencia `define`.

- Se permitirá la creación de eventos, en donde se le asignan al usuario/o a un grupo/s, mediante la declaración event.
- Se incluirá la posibilidad de generar gráficos que representen las tareas de los usuarios en cuestión, mediante la sentencia generate. Se podrán utilizar los siguientes gráficos:
 - Cronogramas Semanales: Visualizaciones claras de la distribución de tareas y eventos durante una semana.
 - Calendarios Mensuales: Representación mensual de eventos y tareas asignadas.
 - Calendarios Anuales: Representación anual de eventos y tareas asignadas.

3. Implementación

3.1. Frontend

En esta sección del trabajo se enfocó en definir el lenguaje que se iba a utilizar para generar los diferentes calendarios y luego poder procesar el texto ingresado como nuestro lenguaje.

Se definieron los comandos y qué se obtendrían de los mismos, permitiéndonos esquematizar el procesamiento de cada uno de los tokens que lo componen. Una vez que se concretó cómo se realizaría el mismo, se definió la gramática.

En cuanto a la implementación del frontend, el mismo se dividió en dos secciones. Para el análisis sintáctico del lenguaje se utilizó Flex (Fast Lexical Analyzer), transformando el texto ingresado en el programa en tokens. Posteriormente, mediante la gramática definida se utilizó Bison (GNU Parser generator) para validar los tokens.

En flex se definieron tokens como delimitadores, los días de la semana, hora, fecha, el tipo de calendario a generar y tokens propios de la sintaxis de la gramática como “create” o “generate”.

3.2. Backend

La generación de la salida del programa comienza con la función principal generate. Esta función itera sobre la lista de Generates, la cual provee las especificaciones y los datos que deben ser generados. Por cada uno de los ítems en la lista, se abre un archivo de salida con el nombre definido como el id del generate. A continuación, se llaman a tres funciones:

- **_generatePrologue()**, que se encarga de generar la estructura básica HTML de todos los calendarios.
- **_generateType()**, que es un switch case que, dependiendo del tipo que se desea generar, llama a la función correspondiente.
- **_generateEpilogue()**, que se encarga de cerrar las etiquetas y rellenar el archivo con una lista de los distintos usuarios declarados, junto con sus colores identificadores en el calendario.

Como se mencionó previamente, la función **_generateType()** es la encargada de identificar el tipo de calendario para su generación. Si bien todas las tasks, events, groups y users son manejados de la misma forma dentro de las distintas funciones del tipo de calendario, lo que cambia es la definición de la función JavaScript que lo genera. El proceso de generación consiste en tomar las tasks y events relacionados con el usuario, ya sea mediante declaración explícita (asignándose directamente) o implícita (asignándose a un grupo al que pertenece el usuario), e inyectarlas en el archivo con su formato correspondiente.

Para poder relacionar correctamente a un usuario con sus tasks y events, se implementó el uso de cuatro tablas de símbolos, las cuales están declaradas en el archivo SymbolTables.c. Estas son:

- **generateList**: lista de entradas generate, que contienen una copia del estado de las otras tablas en el momento en que se solicitó hacer un generate. De esta forma, nos aseguramos de generar únicamente las tasks y events de usuarios y grupos que fueron declarados hasta ese momento.
- **usersTableMap**: hashmap que utiliza la librería uthash.h, en la cual se van agregando los distintos usuarios que se crean. Al momento de la creación de un nuevo usuario, se valida que el mismo no se encuentre ya en la tabla (es decir, que no se repitan los IDs). Luego, cada usuario dentro de esta tabla tiene asociadas las tasks, events y grupos a los cuales fue asignado.
- **groupsTableMap**: similar a usersTableMap, pero en este caso guarda a los grupos junto con sus tasks y events asociados.
- **codeBlocksTableMap**: hashmap que guarda la información de los bloques de código definidos mediante el programa. Este cuenta con una estructura en la cual se guarda la tabla de usuarios, la tabla de grupos y la lista de generates propia del bloque de código.

Una vez que todos los archivos hayan sido generados correctamente, la función principal finaliza su ejecución. Luego, todas las estructuras son liberadas en EntryPoint.c, completando así el proceso de salida.

3.3. Dificultades encontradas

Principalmente, nos encontramos con un problema en el desarrollo del trabajo cuando debíamos utilizar una librería para almacenar los datos generados. Entre ellos se encontraban datos como los usuarios o los grupos, ya que los mismos podían ser referenciados posteriormente en el código y se debía realizar algún tipo de validación.

Para ello, se utilizó la librería mencionada para almacenar los datos en hashmap y posteriormente obtenerlos del mismo.

4. Futuras extensiones

Al generar la salida del programa en un lenguaje como Javascript, el mismo nos permite acceso a múltiples librerías que nos permiten esquematizar la información de eventos, tareas, usuarios y grupos en múltiples formatos. Es por eso que a futuro se podría implementar otras salidas como diagramas de Gantt, listas. Incluso se le podría permitir al usuario seleccionar la librería que se desea utilizar.

5. Conclusiones

Tras haber finalizado el trabajo, es impactante poder decir que se logró interpretar un lenguaje nuevo y ver en el proceso cómo era posible formatear la información como quisiéramos. Fue muy interesante poder poner en práctica lo que nos enseñó la materia y ver cómo se reflejan las mismas en el ámbito de la informática y los lenguajes de programación.

El uso de lenguajes como **Flex** y **Bison**, nos permitió concentrarnos en lo que era realmente importante y poder implementar la gramática y sintaxis del mismo lo mejor posible, así mismo cómo la base del proyecto y los ejemplos proporcionados en clase.

En conclusión, este proyecto no solo permitió aplicar los conceptos teóricos aprendidos en clase, sino que también ofreció una oportunidad para profundizar en la comprensión de cómo se construyen e implementan los lenguajes de programación desde cero.

6. Referencias

- FullCalendar LLC. (n.d.). *FullCalendar documentation*. Retrieved October 20, 2024, from <https://fullcalendar.io/docs>
- Hanson, T. (n.d.). *uthash user guide*. Retrieved November 2, 2024, from <https://troydhanson.github.io/uthash/userguide.html>